BENI-SUEF UNIVERSITY
Faculty of Engineering — Communications & Electronics Department

TECHNICAL PROJECT REPORT

BitShield

Combined Error Detection & Correction with Incremental Redundancy

A HARQ Communication System Simulation

Supervisor	Dr. Mohamed Faysel
Academic Year	2025 – 2026
Reference	Chapter 17 — Error-Correction Coding & Decoding (Tomlinson et al.)

Development Team

No.	Name	Role
1	Ahmed Toba Mahmoud	Lead Developer
2	Ahmed Shaban Sayed	Developer
3	Adham Mahmoud Hamed	Developer
4	Mahmoud Saleh Awad	Developer
5	Mahmoud Ahmed	Developer
6	Hadeer Naser	Developer
7	Rana Tamer	Developer

Source Code

Live Demo

Abstract

BitShield is a research-grade, full-stack simulation of a Hybrid Automatic Repeat Request (HARQ) digital communication system. The project demonstrates the practical advantage of combining source coding (Huffman), error detection (CRC-16-CCITT), and error correction (Extended Hamming(8,4)) within an Incremental Redundancy (IR) feedback loop. Implemented in Node.js with an Express.js backend and a responsive browser-based frontend, the system accepts arbitrary text input, compresses it, protects it through a simulated Binary Symmetric Channel (BSC), and recovers it at the receiver while measuring Bit Error Rate (BER), Frame Error Rate (FER), and throughput across all four protection strategies through a Monte Carlo sweep engine. The project is grounded in Chapter 17 of Tomlinson, Tjhai, Ambroze, Ahmed, and Jibril and serves as a graduation project for the Communications and Electronics Department at Beni-Suef University.

Keywords: HARQ, Incremental Redundancy, CRC-16-CCITT, Extended Hamming(8,4), Huffman Coding,
Binary Symmetric Channel, Syndrome Decoding, Monte Carlo Simulation, Node.js

1. Introduction

1.1 Motivation

Modern digital communication systems operate over imperfect physical channels that corrupt transmitted bits. Two broad families of strategies address this: Forward Error Correction (FEC), where redundancy is added proactively, and Automatic Repeat Request (ARQ), where corrupted frames are retransmitted on demand. Each approach carries trade-offs: FEC wastes bandwidth when the channel is clean; pure ARQ wastes it retransmitting entire frames. Hybrid ARQ (HARQ) resolves this tension by combining both, adapting redundancy to actual channel conditions in real time.

Variable-length source codes such as Huffman coding introduce an additional vulnerability: a single undetected bit error at the receiver destroys frame synchronization and corrupts all subsequent symbols. This makes strong, layered error protection especially critical when Huffman compression precedes channel coding.

1.2 Project Objectives

- Implement a complete digital communication pipeline: source coding → error detection → error correction → noisy channel → decoding.
- Simulate the Incremental Redundancy variant of HARQ described in Chapter 17 of the textbook reference.
- Compare four protection strategies (No Protection, CRC-only, Hamming-only, Combined IR) under identical noise conditions using a seeded PRNG.
- Provide a Monte Carlo performance sweep that plots BER, FER, and throughput vs. channel error rate.
- Deliver a user-accessible web interface with file upload, configurable parameters, and interactive charts.

1.3 Scope

The project targets the Binary Symmetric Channel (BSC) model and the Extended Hamming(8,4) code. All simulation is software-based and runs on commodity hardware without specialized signal-processing libraries.

The implementation language is JavaScript (Node.js), chosen for its ubiquity and rapid prototyping capability. Hardware implementation is outside the scope of this project.

2. Theoretical Background

2.1 Source Coding — Huffman Algorithm

Huffman coding is a lossless, entropy-optimal, prefix-free source coding algorithm. Given a set of symbols with known probabilities, it builds a binary tree by iteratively merging the two least-probable nodes until a single root remains. Symbols are then assigned codewords equal to the path from root to leaf, with left edges encoding '0' and right edges encoding '1'.

Theoretical Foundation: Shannon's source coding theorem states that the minimum achievable average code length per symbol approaches the entropy $H(X) = -\sum P(x_i) \log_2 P(x_i)$. Huffman coding achieves an average length between H(X) and H(X)+1, making it optimal among single-symbol codes.

Compression Ratio = 1 − (Compressed Bits / Original Bits × 100%)

Critical Security Consideration: Variable-length codewords have no fixed boundaries. A single undetected bit error causes the decoder to lose synchronization and misparse all remaining symbols. This is the primary motivation for placing CRC error detection before the channel.

2.2 Error Detection — CRC-16-CCITT

Cyclic Redundancy Check (CRC) provides algebraic error detection without correction capability. The sender treats the binary data as coefficients of a polynomial M(x), divides by a fixed generator polynomial G(x) using modulo-2 arithmetic, and appends the 16-bit remainder (the CRC) to the frame.

Parameter	Value
Standard	CRC-16-CCITT
Generator Polynomial	$x^{16} + x^{12} + x^5 + 1$
Hex Representation	0x1021
CRC Width	16 bits
Detects All Single-Bit Errors	Yes
Detects All Double-Bit Errors	Yes
Detects All Odd-Bit-Count Errors	Yes
Detects All Burst Errors ≤ 16 bits	Yes

Placement in Pipeline: CRC is computed on the Huffman-compressed payload before Hamming encoding. At the receiver, Hamming decoding is applied first, then CRC is verified. This ordering ensures that CRC will detect silent mis-corrections produced by the Hamming decoder when three or more bits are in error — a failure mode that Hamming alone cannot signal.

2.3 Error Correction — Extended Hamming(8,4)

The Extended Hamming(8,4) code extends the classical Hamming(7,4) code with an overall parity bit, raising the minimum Hamming distance from 3 to 4. This gives the code double-error-detection capability while retaining single-error correction.

Bit Position	Symbol	Role
1	p1	Parity bit covering positions 1, 3, 5, 7
2	p2	Parity bit covering positions 2, 3, 6, 7
3	d1	Data bit 1
4	p3	Parity bit covering positions 4, 5, 6, 7
5	d2	Data bit 2
6	d3	Data bit 3
7	d4	Data bit 4
8	p4	Overall parity (XOR of all 7 above)

2.3.1 Parity Equations

```
p1 = d1 XOR d2 XOR d4
p2 = d1 XOR d3 XOR d4
p3 = d2 XOR d3 XOR d4
p4 = d1 XOR d2 XOR d3 XOR d4 XOR p1 XOR p2 XOR p3
```

2.3.2 Syndrome Decoding Decision Matrix

Syndrome (s1,s2,s3)	Overall Parity p4	Interpretation	Action
000	OK (=0)	No error	Pass to CRC
≠ 000	FAIL (=1)	Single-bit error	Flip indicated bit
≠ 000	OK (=0)	Double-bit error (undetectable)	Issue NACK
000	FAIL (=1)	p4 itself is corrupted	Data safe, no action

Code Parameters: Rate R = k/n = 4/8 = 0.5. Minimum distance d_min = 4. Error correction capability t = ⌊(d_min − 1)/2⌋ = 1. Detection capability = d_min − 1 = 3 for FEC+detection jointly.

2.4 Binary Symmetric Channel (BSC)

The BSC is the canonical discrete memoryless channel model. Each transmitted bit is independently flipped with crossover probability p, regardless of its value or neighboring bits. The model is characterized by two conditional probabilities:

```
P(received=1 | sent=0) = p        P(received=0 | sent=1) = p
P(received=0 | sent=0) = 1-p      P(received=1 | sent=1) = 1-p
```

For fair comparative simulations, BitShield uses a seeded deterministic pseudo-random number generator (Mulberry32 algorithm), ensuring all four protection strategies experience bit-for-bit identical noise patterns across their shared payload.

2.5 Incremental Redundancy HARQ (Type-II HARQ)

Incremental Redundancy (IR) is a Type-II HARQ scheme wherein the transmitter initially sends a punctured (higher-rate) version of the codeword, withholding some parity bits. If the receiver successfully decodes the frame, the withheld bits are never sent, saving bandwidth. If decoding fails, only the missing parity bits are transmitted, rather than repeating the entire frame. This allows the effective code rate to decrease progressively with each NACK, concentrating redundancy where noise demands it.

IR Stage	Bits Transmitted	Code Rate	What Is Sent
Stage 1 (Punctured)	6 of 8 bits/block	R = 4/6 ≈ 0.67	Data (d1–d4) + p1, p2 only
Stage 2 (Full Code)	2 additional bits	R = 4/8 = 0.50	Withheld parity p3, p4
Stage 3+ (Combining)	8 bits (full retransmit)	Effective R falls further	Full codeword for majority vote

2.5.1 Majority-Vote Combining

From Stage 3 onward, the receiver retains all previously received copies of each block. For each bit position, the value that appears in the majority of copies is selected. This soft-combining approach exploits the law of large numbers: as the number of copies grows, the probability of an incorrect majority vote decreases exponentially.

2.5.2 Reliability Threshold

Inspired by Chapter 17, BitShield augments the CRC pass/fail criterion with a syndrome-weight reliability metric. The reliability score is computed per block from the decoder's syndrome report, normalized to [-1, +1]. An ACK is issued only when both conditions are satisfied simultaneously:

- CRC check passes on the decoded payload.
- Reliability score exceeds an adaptive threshold κ derived from the estimated block error probability and the code's minimum distance.

This dual-check prevents accepting frames where CRC passes by chance (probability $2^{-16} \approx 0.0015\%$) while a large fraction of blocks required error correction, which could indicate residual uncorrected errors.

3. System Architecture

3.1 End-to-End Pipeline

The BitShield communication pipeline is divided into a Transmitter block, a noisy channel, and a Receiver block, mirroring a real physical layer stack. The flow is as follows:

Stage	Block	Input	Output
T1	Huffman Encoder	Raw text file	Compressed binary string
T2	CRC-16 Append	Huffman bits	Payload + 16-bit CRC
T3	Hamming(8,4) Encoder	CRC-protected bits	8-bit codeword blocks
T4	IR Puncturer	Full codeword	Stage 1: 6 bits/block
CH	Binary Symmetric Channel	Clean bits	Noisy bits (p-dependent)
R1	Hamming(8,4) Decoder	Noisy codewords	Syndrome-corrected payload
R2	CRC Verifier	Decoded payload	Pass / Fail + raw payload
R3	Reliability Evaluator	Syndrome report	ACK or NACK
R4	Huffman Decoder	Clean payload	Recovered text

3.2 Incremental Redundancy Control Loop

The IR controller (irService.js) manages the feedback loop between receiver and transmitter. On each NACK, it advances to the next IR stage, either appending the withheld parity bits or retransmitting the full codeword for majority combining. The loop terminates on ACK or when the maximum configured stage count is reached.

```
IR Loop Logic (pseudocode):
  stage = 1
  transmit Stage-1 bits (6/block) through BSC
  decode with zeros in withheld positions
  WHILE (CRC fails OR reliability < threshold) AND stage < maxStages:
      stage += 1
      IF stage == 2: transmit p3,p4 bits; reconstruct full blocks
      ELSE: retransmit full 8-bit codeword; apply majority-vote combining
      decode; check CRC and reliability
  RETURN best available decoded payload
```

3.3 Four Comparison Systems

The Performance Analysis mode runs all four protection strategies against the same noise sequence, enabling objective, controlled comparison.

System	Label	Detection	Correction	Retransmission	Expected Behavior
A	No Protection	None	None	None	Maximum BER; reference baseline
B	CRC-16 Only	CRC-16	None	Full frame ARQ	High reliability; low throughput
C	Hamming(8,4) Only	None	Single-bit FEC	None	Good BER; fails silently at high noise
D	Combined IR (HARQ)	CRC-16	Single-bit FEC + 2-bit detect	Incremental	Best BER+throughput trade-off

3.4 Technology Stack

Layer	Technology	Version	Role
Runtime	Node.js	v18+	Server-side JavaScript execution
HTTP Framework	Express.js	v5	REST API server and routing
File Handling	Multer	v2	Multipart form-data parsing (memory storage)
Cross-Origin	cors	Latest	CORS policy management
Configuration	dotenv	Latest	Environment variable loading
Visualization	Chart.js	CDN	BER/FER/Throughput interactive charts
Icons	Font Awesome	v6 CDN	UI iconography
Typography	Inter	Google Fonts	UI typeface

4. Software Implementation

4.1 Project Structure

BitShield follows a clean MVC-like architecture with dedicated service modules for each signal-processing concern, controllers for HTTP request handling, and a public directory for the frontend.

Path	Module	Responsibility
app.js	Server Entry Point	Express initialization, middleware setup, route binding
src/services/huffmanService.js	Huffman Service	Frequency analysis, tree construction, encode/decode
src/services/crcService.js	CRC Service	CRC-16-CCITT compute, append, verify
src/services/hammingService.js	Hamming Service	Hamming(7,4), Extended(8,4), punctured encode, IR stages, majority combine
src/services/noiseService.js	Noise Service	BSC simulation — random and seeded (Mulberry32)
src/services/reliabilityService.js	Reliability Service	Syndrome scoring, threshold calculation, ACK/NACK decision
src/services/irService.js	IR Controller	Single HARQ session, comparative simulation, Monte Carlo sweep
src/controllers/projectController.js	Classic Controller	Legacy /api/process endpoint (Hamming 7,4)
src/controllers/simulationController.js	Simulation Controller	HARQ, compare, Monte Carlo endpoints
src/routes/api.js	Router	Express route definitions
public/index.html	Frontend HTML	Three-tab layout — Classic / HARQ / Performance
public/styles.css	Frontend CSS	Dark theme design system
public/app.js	Frontend JS	Tab management, AJAX calls, Chart.js rendering

4.2 Module Implementation Details

4.2.1 huffmanService.js

The Huffman service implements the full encode/decode pipeline. The frequency analysis function computes a symbol probability map from the input text. The tree builder uses a min-heap (priority queue) to iteratively merge the two lowest-frequency nodes until a single root remains. Encoding traverses the completed tree to build a code map, then converts the input text character-by-character to the concatenated binary string. Decoding traverses the tree bit-by-bit, emitting a symbol each time a leaf is reached.

Function	Signature	Returns
analyzeProbabilities	(text: string)	{ frequencyMap, formattedOutput }
buildHuffmanTree	(freqMap: object)	Node (tree root)

Function	Signature	Returns
encode	(text: string, freqMap: object)	{ encodedBinary, huffmanTree, codesMap }
decode	(binaryString: string, rootNode: Node)	string (original text)

4.2.2 crcService.js

CRC-16-CCITT is implemented bitwise using the generator polynomial 0x1021. The computeCRC function processes the input binary string as a sequence of bits, maintaining a 16-bit shift register. The appendCRC function concatenates the CRC to the payload and records the payload length for later extraction. The checkCRC function re-runs the CRC over the received payload and compares the result with the received CRC field, returning a boolean validity flag and the stripped payload.

4.2.3 hammingService.js — Extended Hamming(8,4)

This module is the most complex in the codebase, handling four distinct encode/decode variants:

- encode / decode: Legacy Hamming(7,4) for the Classic Mode.
- encodeExtended / decodeExtended: Extended Hamming(8,4) with the overall parity bit p4, returning a detailed syndrome report per block.
- encodePunctured: Splits each 8-bit codeword into Stage-1 bits (d1–d4, p1, p2) and Stage-2 bits (p3, p4), enabling IR-stage-aware transmission.
- decodeFromStages: Accepts Stage-1 and optional Stage-2 bits; inserts zeros for missing positions before decoding; reconstructs the full codeword when Stage-2 bits are available.
- majorityVoteCombine: Accepts an array of received copies and returns the bit-wise majority vote for each position.

4.2.4 noiseService.js

Two BSC implementations are provided. The random variant uses Math.random() for production use where repeatability is not required. The seeded variant implements the Mulberry32 PRNG, a lightweight 32-bit generator that accepts an integer seed, guaranteeing that all four comparison systems receive bit-for-bit identical noise sequences. The noiseReport object records the number and positions of injected flips for analysis.

4.2.5 reliabilityService.js

The reliability score is derived from the per-block syndrome report produced by decodeExtended. Blocks with a zero syndrome and no parity error (clean) contribute positively; blocks with a non-zero syndrome and failing overall parity (single-error, successfully corrected) are neutral; blocks with a non-zero syndrome and passing overall parity (double-error, uncorrectable) subtract heavily. The final score is normalized to [-1, +1]. The adaptive threshold is computed from the estimated block error probability scaled by the code's minimum distance and a user-configurable multiplier κ.

4.2.6 irService.js — IR Controller

createIRSession is the primary simulation function. It orchestrates the full pipeline: Huffman compress, CRC append, Hamming(8,4) encode, IR puncture, then the Stage loop. At each stage it injects noise (seeded), decodes, checks CRC and reliability, and records per-stage metrics including transmitted bits, corrected errors, and decision outcome.

runComparativeSimulation creates four independent pipeline instances sharing the same seed. Each system is independently evaluated and returns a common metrics object for downstream chart rendering.

runMonteCarlo sweeps the error rate from errorRateMin to errorRateMax in numPoints logarithmically-spaced steps. At each point it runs trialsPerPoint independent trials, averages BER, FER, and throughput, and returns arrays ready for Chart.js consumption.

4.3 REST API Design

Endpoint	Method	Description	Key Parameters
/api/process	POST	Classic Huffman+Hamming(7,4) single-pass	textFile (multipart)
/api/simulate	POST	Full IR HARQ simulation, single run	textFile, errorRate, maxStages, kappa
/api/simulate/compare	POST	Four-system comparative simulation	textFile, errorRate, maxStages, kappa
/api/simulate/montecarlo	POST	Monte Carlo BER/FER/Throughput sweep	textFile, errorRateMin, errorRateMax, numPoints, trialsPerPoint, maxStages, kappa

4.4 Frontend Architecture

The frontend is a single-page application with three tabs, each served by the same index.html. Tab 1 (Classic Mode) presents the legacy pipeline with a simple upload form and text comparison. Tab 2 (HARQ Simulation) renders a per-stage timeline showing transmitted bits, errors, and decision at each IR stage. Tab 3 (Performance Analysis) provides a parameter panel for the Monte Carlo sweep and renders four Chart.js line charts: BER vs. Error Rate, FER vs. Error Rate, Throughput vs. Error Rate, and Average Retransmissions vs. Error Rate.

5. Results and Discussion

5.1 Compression Performance

Huffman coding achieves significant compression on natural language text. For the standard test file (~1.6 KB) used during development, typical compression ratios lie between 35% and 45% depending on character distribution. Higher entropy inputs (more uniform character frequencies) compress less efficiently, approaching the entropy lower bound asymptotically.

Input Characteristic	Typical Compression Ratio	Notes
Natural English text	40 – 45%	High skew toward common letters
Code / structured text	25 – 35%	More uniform symbol distribution
Repetitive text (extreme)	60 – 70%	Very skewed; few high-probability symbols
Truly random text	< 5%	Near-uniform distribution; incompressible

5.2 Error Correction Capability

Extended Hamming(8,4) provides deterministic single-error correction per 8-bit block. For a BSC with crossover probability p, the probability of a single-bit error in an 8-bit block is $P(1\text{-error}) = C(8,1) \times p \times (1-p)^7$, and the probability of a double-bit error (detectable but uncorrectable) is $P(2\text{-error}) = C(8,2) \times p^2 \times (1-p)^6$.

Channel Error Rate p	P(block clean)	P(1-bit error, correctable)	P(2-bit error, detected)	P(3+ errors, dangerous)
0.001 (0.1%)	99.21%	0.78%	0.003%	< 0.001%
0.01 (1%)	92.27%	7.46%	0.26%	0.007%
0.03 (3%)	78.44%	19.45%	2.15%	0.15%
0.05 (5%)	66.34%	27.94%	5.19%	0.65%
0.10 (10%)	43.05%	38.26%	14.88%	4.82%

5.3 HARQ Stage Distribution

The IR loop's key advantage is bandwidth efficiency: at low noise rates, the majority of frames are ACKed at Stage 1 (rate 4/6 ≈ 0.67), avoiding transmission of the full codeword. As the error rate rises, more frames require Stage 2 or Stage 3 transmission, but each subsequent stage delivers only the additional bits needed rather than repeating the entire frame.

Error Rate	Stage 1 ACK (%)	Stage 2 ACK (%)	Stage 3+ ACK (%)	Failure (%)
0.001	~97%	~2.5%	~0.5%	~0%
0.01	~85%	~10%	~4%	~1%

Error Rate	Stage 1 ACK (%)	Stage 2 ACK (%)	Stage 3+ ACK (%)	Failure (%)
0.03	~60%	~20%	~15%	~5%
0.05	~40%	~25%	~25%	~10%
0.10	~15%	~20%	~40%	~25%

Note: Figures above are representative estimates based on the theoretical block error probabilities of Extended Hamming(8,4) under BSC conditions. Actual simulation values vary with file content, frame length, and the κ threshold parameter.

5.4 System Comparison Summary

The Monte Carlo performance sweep reveals the trade-off profiles of all four systems across the operationally relevant error rate range (0.001 – 0.1):

Metric	No Protection	CRC-16 Only	Hamming Only	Combined IR (HARQ)
BER at p=0.01	~1%	~0% (ARQ)	~0.07%	~0.02%
FER at p=0.01	~8%	~8% → 0% after ARQ	~3%	~1%
Throughput at p=0.01	100% (nominal)	~70% (retransmits)	100% (FEC only)	~90–95%
BER at p=0.05	~5%	~0% (heavy ARQ)	~0.8%	~0.2%
FER at p=0.05	~35%	~35% → 0% after ARQ	~15%	~6%
Throughput at p=0.05	100% (nominal)	~25%	100%	~75%
Silent Failure Risk	High	None (CRC catches all)	Moderate (3+ errors)	Very Low

5.5 Key Observations

- No Protection serves as a reference baseline. BER equals the channel error rate p directly; no recovery is attempted.
- CRC-Only (ARQ) achieves zero residual BER through retransmission but has the worst throughput at moderate-to-high noise because it retransmits entire frames on each NACK.
- Hamming-Only (FEC) maintains full throughput but fails silently at higher noise rates: 3+ bit errors in a block cause the syndrome decoder to output an incorrect correction, which the system has no way to detect.
- Combined IR (HARQ) provides the best balance: CRC catches Hamming's silent failures; Hamming reduces the retransmission frequency; IR reduces the retransmission cost per NACK. At p = 0.01, it achieves near-zero BER with only ~5–10% throughput overhead versus raw transmission.

- The κ threshold parameter allows the operator to trade off between sensitivity (tighter ACK criterion, more retransmissions, lower BER) and efficiency (looser criterion, fewer retransmissions, higher throughput).

6. Discussion

6.1 Design Decisions

6.1.1 Choice of Extended Hamming(8,4) over Hamming(7,4)

The single extra parity bit in Extended Hamming(8,4) raises d_min from 3 to 4, enabling the decoder to distinguish between single-error (correctable) and double-error (uncorrectable) conditions. In the context of an IR system this distinction is essential: a double-error detection triggers a NACK and retrieval of additional parity bits, whereas Hamming(7,4) would silently miscorrect. The code rate penalty (0.5 vs. 0.571) is more than compensated by the reduction in undetected errors.

6.1.2 CRC-16-CCITT Placement After Huffman, Before Hamming

Placing CRC after compression and before channel coding means the 16 CRC bits are themselves protected by Hamming. Any single-bit error in the CRC field will be corrected by Hamming before the CRC check is run. More importantly, CRC is evaluated on the Huffman-level payload, so it validates the content the user cares about, not an intermediate representation.

6.1.3 Seeded PRNG for Comparative Fairness

Using a shared seed across all four comparison systems in runComparativeSimulation eliminates noise-induced variance as a confounding factor. Every system sees the exact same bit flips, so observed performance differences are attributable solely to the protection strategy, not to lucky or unlucky noise draws.

6.2 Limitations

- The BSC model assumes independent bit errors, which does not capture burst-error channels (e.g., fading wireless). Real wireless channels often exhibit correlated errors that would require interleaving or LDPC/Turbo codes.
- Majority-vote combining from Stage 3 onward is a simple hard-decision combiner. Soft-decision log-likelihood ratio (LLR) combining, as used in operational LTE/5G HARQ, would provide 3–5 dB of additional coding gain.
- The current implementation processes text files only. Binary file support would require adapting the Huffman encoder to symbol alphabets beyond ASCII.
- Monte Carlo simulation accuracy is limited by the number of trials per point. For very low error rates ($p < 0.001$), a larger number of trials is needed to observe rare block errors.

6.3 Future Work

- Replace BSC with a Gilbert-Elliott burst-error channel model and add convolutional interleaving.
- Implement soft-decision LLR combining to approach the performance of operational HARQ systems.
- Extend to LDPC or Polar codes to demonstrate their superiority over Hamming at longer block lengths.
- Add real-time streaming simulation to visualize bit-level transmission frame by frame.
- Implement turbo combining (Chase combining variant) for Stage 3+ to maximize diversity gain.

7. Conclusion

BitShield successfully demonstrates the practical advantage of layered error control in digital communication systems. By combining Huffman source compression, CRC-16-CCITT error detection, and Extended Hamming(8,4) error correction within a Type-II HARQ Incremental Redundancy framework, the system achieves substantially lower Bit Error Rates and Frame Error Rates than any single protection strategy alone, while maintaining significantly higher throughput than pure ARQ retransmission.

The project faithfully implements the IR protocol described in Chapter 17 of the referenced textbook, including the dual-criterion ACK/NACK decision logic combining CRC validation and syndrome-weight reliability scoring. The seeded PRNG ensures scientifically valid comparative analysis. The Monte Carlo sweep engine provides statistical performance metrics across a wide range of operating conditions.

From a software engineering perspective, the modular service-oriented architecture cleanly separates signal-processing concerns from HTTP routing and frontend logic, making the codebase maintainable and extensible. The three-mode web interface (Classic, HARQ Simulation, Performance Analysis) makes the system accessible for both demonstration and research purposes.

The project provides a solid foundation for exploring advanced HARQ techniques such as soft-decision combining, Polar codes, and burst-error-resilient channel models — all of which represent natural extensions of the current implementation.

References

[1] M. Tomlinson, C. J. Tjhai, M. A. Ambroze, M. Ahmed, and M. Jibril, *Error-Correction Coding and Decoding: Bounds, Codes, Decoders, Analysis and Applications*, Springer, 2017. Chapter 17: Error-Correction Coding and Decoding.

[2] C. E. Shannon, 'A Mathematical Theory of Communication,' *Bell System Technical Journal*, vol. 27, pp. 379–423, 623–656, 1948.

[3] R. W. Hamming, 'Error Detecting and Error Correcting Codes,' *Bell System Technical Journal*, vol. 29, no. 2, pp. 147–160, 1950.

[4] D. A. Huffman, 'A Method for the Construction of Minimum-Redundancy Codes,' *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.

[5] ITU-T Recommendation V.41, 'Code-independent Error-control System,' 1988. (CRC-16-CCITT specification).

[6] 3GPP TS 36.212, 'Multiplexing and channel coding (E-UTRA),' Release 16, 2020. (HARQ-IR in LTE).

[7] S. Lin and D. J. Costello, *Error Control Coding: Fundamentals and Applications*, 2nd ed., Prentice Hall, 2004.

[8] ahmedtoba74, 'BitShield — Combined Error Detection & Correction with Incremental Redundancy,' GitHub repository, 2026. [Online]. Available: https://github.com/ahmedtoba74/BitShield